

CAB340
Assessment 3

Eugene Martens
N10318313

Q1

a)

i. $a \equiv b \pmod n \Leftrightarrow a - b = ln$

thus:

$$p^{-1}p \equiv 1 \pmod q \Leftrightarrow p^{-1}p - 1 = lq$$

ii. given: $p^{-1}p - 1 = lq$

where: $p^{-1}p + (-l)q = 1$

and $\gcd(p, q) = 1 = pm + qn$

gives: $pp^{-1} + q(-l) = 1$ where $m = p^{-1}$ & $n = -l$

thus:

$$p^{-1} \text{ exist's (mod } q)$$

b)

$$\gcd(96, 35) = 1$$

j	a_j	b_j	q_j	r_j	$a_j = b_j x q_j + r$
0	96	35	2	26	$96 = 25 \times 2 + 26$
1	35	26	1	9	$25 = 26 \times 1 + 9$
2	26	9	2	8	$26 = 9 \times 2 + 8$
3	9	8	1	1	$9 = 8 \times 1 + 1$
4	8	1	8	0	$8 = 1 \times 8 + 0$

c)

j	a_j	b_j	q_j	m_j	n_j	$a_j * m_j + b_j * n_j = \gcd(a, b)$
3	9	8	1	1	-1	$9 \times 1 + 8 \times -1 = 1$
2	26	9	2	-1	3	$26 \times -1 + 9 \times 3 = 1$
1	35	26	1	3	-4	$35 \times 3 + 26 \times -4 = 1$
0	96	35	2	-4	11	$96 \times -4 + 35 \times 11 = 1$

d)

Bezout's Coefficients	$a_j * m_j + b_j * n_j = \gcd(a, b)$
1, -1	$9 \times 1 + 8 \times -1 = 1$
-1, 3	$26 \times -1 + 9 \times 3 = 1$
3, -4	$35 \times 3 + 26 \times -4 = 1$
-4, 11	$96 \times -4 + 35 \times 11 = 1$

$$96^{-1} \equiv -4 \equiv 31 \pmod{35}, \quad 35^{-1} \equiv 11 \pmod{96}$$

Q2 Calculated using code from "RSA_Assessment.py (v 3.10)" that is included with submission

a) p: 2027, q: 2593, $\lambda(n) (\text{lcm}(p-1, q-1))$: 2625696, d: 1373441, e: 65, n: 5256011

i) Public key (e, n): (65, 5256011)
ii) Private key (d, n): (1373441, 5256011)

Extra credit: calculating private key (d, n)

p - 1: 2026, q - 1: 2592, $\phi(n) (p-1)(q-1)$: 5251392, e: 65

$$e \cdot d \pmod{\phi(n)} = 1 \mid 65 \cdot d \pmod{5251392} = 1$$

$$ax + by = \text{gcd}(a, b), \text{ where } a: 5251392 \text{ \& } b: 65$$

$$\text{gcd}(5251392, 65)$$

j	a_j	b_j	q_j	r_j	$a_j = b_j x q_j + r$
0	5251392	65	80790	42	$5251392 = 65 \cdot 80790 + 42$
1	65	42	1	23	$65 = 42 \cdot 1 + 23$
2	42	23	1	19	$42 = 23 \cdot 1 + 19$
3	23	19	1	4	$23 = 19 \cdot 1 + 4$
4	19	4	4	3	$19 = 4 \cdot 4 + 3$
5	4	3	1	1	$4 = 3 \cdot 1 + 1$

j	a_j	b_j	q_j	m_j	n_j	$a_j * m_j + b_j * n_j = \text{gcd}(a, b)$
5	4	3	1	1	-1	$4 \cdot 1 + 3 \cdot -1 = 1$
4	19	4	4	-1	5	$19 \cdot -1 + 4 \cdot 5 = 1$
3	23	19	1	5	-6	$23 \cdot 5 + 19 \cdot -6 = 1$
2	42	23	1	-6	11	$42 \cdot -6 + 23 \cdot 11 = 1$
1	65	42	1	11	-17	$65 \cdot 11 + 42 \cdot -17 = 1$
0	5251392	65	80790	-17	1373441	$5251392 \cdot -17 + 65 \cdot 1373441 = 1$

$$5251392^{-1} \equiv -17 \equiv 48 \pmod{65}, 65^{-1} \equiv 1373441 \pmod{5251392}$$

where $65^{-1} \equiv 1373441 \pmod{5251392} = d$ for the private key

verifying d & e for a)

m: 999999

encrypt

$$c: m^e \pmod{n} = 999999^{65} \pmod{5256011} = 5218520$$

Decrypt

$$m: c^d \pmod{n} = 5218520^{1373441} \pmod{5256011} = 999999$$

b) message: 1024, c: $\text{message}^{65} \pmod{5256011}$

Square-multiply-algorithm: $x^m \pmod{n}$,

m: 65, x: 1024, p: 5256011

Step 1: find binary representation of m or 65
1000001

Step 2: reading left to right, where the first value
 $h_0 = x$

1:1 $x \pmod{p}$: 1024

2:0 $x - s - x^2 \pmod{p}$: 1048576

3:0 $x - s - x^2 - s - x^4 \pmod{p}$: 1430675

4:0 $x - s - x^2 - s - x^4 - s - x^8 \pmod{p}$: 3615939

5:0 $x - s - x^2 - s - x^4 - s - x^8 - s - x^{16} \pmod{p}$: 4207791

6:1 $x - s - x^2 - s - x^4 - s - x^8 - s - x^{16} - x^{32} \pmod{p}$: 1324861

$$c: 658626$$

c) p: 2027, q: 2593, n: 5256011 d: 1373441

$$d \pmod{p-1} = 1373441 \pmod{2026} = 1839$$

$$d \pmod{q-1} = 1373441 \pmod{2592} = 2273$$

$$M \pmod{p-1} = C^{d \pmod{p-1}} \pmod{p} = 658626^{1839} \pmod{2027} = 1024$$

$$M \pmod{q-1} = C^{d \pmod{q-1}} \pmod{q} = 658626^{2273} \pmod{2593} = 1024$$

$$q1 = 2593^{-1} \pmod{2027} = 727$$

$$q2 = 2027^{-1} \pmod{2593} = 1663$$

$$\begin{aligned} M &= 1024 * 2593 * 727 + 1024 * 2027 * 1663 \pmod{5256011} \\ &= 1930353664 + 3451802624 \pmod{5256011} \\ &= 5382156288 \pmod{5256011} \\ &= 1024 \end{aligned}$$

Q3.1 Calculated using "DiffieHellman_Assessment.py (v3.10)" included with submission

a) 2, 5, 6, 7, 8 using Fermat's little theorem $g^x \pmod{n} \neq 1$

b)

g	$g^x \pmod{2027} \neq 1$
2	1014
3	1
4	1
5	811
6	338
7	1448
8	1267
9	1
10	1

c) p = 2027, g = 2, a = 123, b = 456

$$\text{Alice(a): } 123, 2^{123} \pmod{2027} = 1561$$

Alice Sends: 1561

$$\text{Bob(b): } 456, 2^{456} \pmod{2027} = 211$$

Bob Sends: 211

d) g_b (sent by bob) : 211 g_a (sent by alice): 1561

$$\text{Alice computes: } g_b^a \pmod{p} = 211^{123} \pmod{2027} = 238$$

$$\text{Bob computes: } g_a^b \pmod{p} = 1561^{456} \pmod{2027} = 238$$

Q3.2

a) $p = 2027, q = 2$

$$\phi(p) = p - 1 = 2026$$

$$\phi(p) / q \pmod{p} = 2026/2 \pmod{2027} = 1013$$

$$A = g_a (\text{Alice's message}) = 1561$$

$$A_2 = g_a^{\phi(p)/q \pmod{p}} = 1561^{1013} \pmod{2027} = 2026 (-1)$$

$$B = g_b (\text{bobs message}) = 211$$

$$B_2 = g_b^{\phi(p)/q \pmod{p}} = 211^{1013} \pmod{2027} = 1$$

c) $A_2 = (-1), B_2 = 1, g_2 = (-1)$

$$p = 2027, g = -1$$

$$\log_g x \equiv n \pmod{p} \Leftrightarrow x = g^n \pmod{p}$$

$$a_2 = \log_{-1} A_2 \equiv n \pmod{p} \Leftrightarrow A_2 = -1^n \pmod{2027}$$

$$\rightarrow 2026 = 2026^n \pmod{2027}$$

$$\rightarrow n = 1$$

$$b_2 = \log_{-1} B_2 \equiv n \pmod{p} \Leftrightarrow B_2 = -1^n \pmod{2027}$$

$$\rightarrow 1 = 2026^n \pmod{2027}$$

$$\rightarrow n = 0$$

b) $g = 2$

$$g_2 = g^{\phi(p)/q \pmod{p}} = 2^{1013} \pmod{2027} = 2026 (-1)$$

operation used to find a generator:

loop g : 2 to 10: i

loop i : 1 to $\phi(p)$: j

if $g^i \pmod{p} = 1$: not a generator

if $g^i \pmod{p} \neq 1$: is a generator

generator's of g^{2025} were found to be a valid for p and from that would aid in an educated guess for g_2

g	$g^i \pmod{p}$	i
✓ 2	1014	2025
x 3	1	1013
x 4	1	1013
✓ 5	811	2025
✓ 6	338	2025
✓ 7	1448	2025
✓ 8	1267	2025
x 9	1	1013
x 10	1	1013

d) $b_2=0, a_2 = 1$

Alice computes:

$$B_2^{a_2} \pmod{2} = 1^1 \pmod{2} = 1$$

Bob computes:

$$A_2^{b_2} \pmod{2} = -1^0 \pmod{2} = 1$$

$$K_2 = 1$$

Extra Credit

Verifying K_2 , K : 238

$$K_2 = K^{\phi(p)/q \pmod{p}} = 238^{1013} \pmod{2027} = 1$$

e) Using **Sophie Germain prime**: $p = 2q + 1$, we find that a prime subgroup $q = 1013$ for 2027

$$p = 2027, a = 123, b = 456, g = 3$$

$$\text{Alice}(g_a): 123, 3^{123} \pmod{2027} = 1587$$

$$\text{Bob}(g_b): 456, 3^{456} \pmod{2027} = 1519$$

Alice computes:

$$g_b^a \pmod{p} = 1519^{123} \pmod{2027} = 1973$$

Bob computes:

$$g_a^b \pmod{p} = 1587^{456} \pmod{2027} = 1973$$

$$g_2 = g^{\phi(p)/q \pmod{p}} = 3^{1013} \pmod{2027} = 1$$

$$A_2 = g_a^{\phi(p)/q \pmod{p}} = 1587^{1013} \pmod{2027} = 1$$

$$B_2 = g_b^{\phi(p)/q \pmod{p}} = 1519^{1013} \pmod{2027} = 1$$

where: $\log_1(A_2, B_2)$ is now undefined

Using the same operation as above, to find a 'g' in subgroup q (1013):

loop g : 2 to 10: i

loop i : 1 to $\phi(q)$: j

if $g^i \pmod{q} = 1$: not a generator

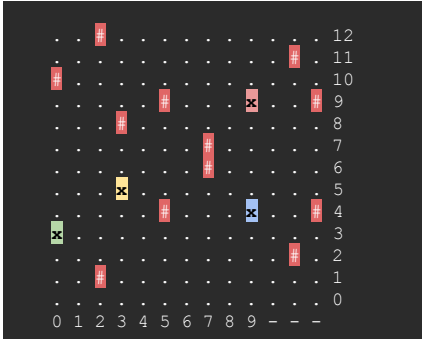
if $g^i \pmod{q} \neq 1$: is a generator

g	$g^i \pmod{q}$	i
x 2	1	92
✓ 3	338	1011
x 4	1	46
✓ 5	608	1011
x 6	1	253
✓ 7	579	1011
x 8	1	92
x 9	1	506
x 10	1	253

f) if a generator is based on the whole of Z^* a multiplicative group it always creates a generator based on the subgroup size of 2 and so by generating through a subgroup q instead, will create the subgroups of 1,2,q (1011) and p-1 (2025), and by picking q as the subgroup for the generator, eliminates the small sub group (2) issue.

Q4.1 Calculated using "ECC_Complete.py (v3.10)" code is displayed and file included with submission

a) $p = 13, a = 5, b = 9$



$G(0,3)$ $H(3,5)$ $G2(9,9)$ $G+H(9,4)$

b) Order of the graph is $(16 + 0) = 17$

c) 10318313: $G:(0,3)$ $H:(3,5)$

$G = (0,3)$

$$9 = 0 + 5 \cdot 0 + 9$$

$$9 = 9 \pmod{13}$$

$$(0^3 + 5 \cdot 0 + 9 - 9) \pmod{13} =$$

$H = (3,5)$

$$25 (12 \pmod{13}) = 3^3 + 5 \cdot 3 + 9$$

$$12 = 17 + 15 + 9$$

$$12 = 41 (12 \pmod{13})$$

$$(3^3 + 5 \cdot 3 + 9 - 5^2) \pmod{13} = 0$$

$$(27 + 15 + 9 - 25) \pmod{13} = 0$$

for d), f) and 4.2 b), c), d) the following functions:

$$x_r: s^2 - x_p - x_q \pmod{p}$$

$$y_r: s(x_p - x_q) - y_p \pmod{p}$$

$$s \text{ for add / sAdd} : (3 \cdot x_p^2 + a) \cdot (2 \cdot y_p) \pmod{p}$$

$$s \text{ for doubling / sDouble: } (y_q - y_p) \cdot (x_q - x_p)^{-1} \pmod{p}$$

Was converted to Python 3.10 (right) using Pycharm IDE and will be included with assessment submission

d) $2G = 2 \cdot (0,3) = (9,9)$

```
pointDoubling(0, 3, 5, 13) : (9, 9)
```

f) $G + H = (0,3) + (3,5)$

```
pointAddition(0, 3, 3, 5, 13) : (9, 4)
```

full code used for 4.1 & 4.2 coded in Python 3.10 using pycharm IDE

```
def modinv2(a, p):
    return pow(a, -1, p)

# Point Addition S
def sAdd(y_p, y_q, x_p, x_q, p):
    return ((y_q - y_p) * modinv2((x_q - x_p), p)) % p

# Point Doubling S
def sDouble(y_p, x_p, a, p):
    return ((3*(x_p**2)+a) * modinv2(2*y_p, p)) % p

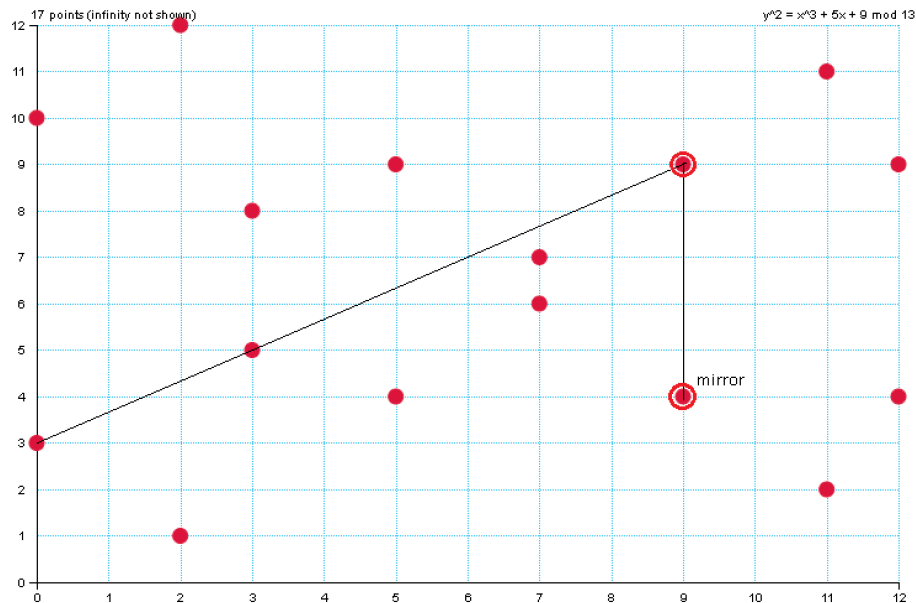
def x_r(sr, x_p, x_q, p):
    return ((sr ** 2) - x_p - x_q) % p

def y_r(s, y_p, x_p, xr, p):
    return ((s * (x_p - xr)) - y_p) % p

def pointAddition(x_p, y_p, x_q, y_q, p):
    sr = sAdd(y_p, y_q, x_p, x_q, p)
    xr = x_r(sr, x_p, x_q, p)
    yr = y_r(sr, y_p, x_p, xr, p)
    return xr, yr

def pointDoubling(x_p, y_p, a, p):
    sr = sDouble(y_p, x_p, a, p)
    xr = x_r(sr, x_p, x_p, p)
    yr = y_r(sr, y_p, x_p, xr, p)
    return xr, yr
```

e) $G + P$: intersect $(9,9)/2G$ and the mirror is $(9, 4)$



4.2 "ECC_Complete.py & ECC_tinyec_example.py (for a) (v3.10)" noted in the question and included with submission

- a) As the order of the curve is prime, any k value $\ast G$, would represent a point on the curve and thus order is 17

```

0 * G = (None, None)
1 * G = (0, 3)
2 * G = (9, 9): calculated above
3 * G = (3, 8)
4 * G = (7, 7)
5 * G = (5, 9)
6 * G = (11, 2)
7 * G = (12, 4)
8 * G = (2, 12)
9 * G = (2, 1)
10 * G = (12, 9)
11 * G = (11, 11)
12 * G = (5, 4)
13 * G = (7, 6)
14 * G = (3, 5)
15 * G = (9, 4)
16 * G = (0, 10)
-----
17 * G = (None, None): cycle starts again
18 * G = (0, 3)
19 * G = (9, 9)

```

Using the code in 4.1 to make the following calculations

- b) $a = 3$

Alice's public point(A): $aG = 3G = (3, 8)$

- c) $b = 6$

Bob's public point(B) : $bG = 6G = (11, 2)$

Console:

```

Alice's Public Point:
(3, 8)
Bob's Public Point:    (11, 2)

```

- d)

Alice: $K = aB = 3 \ast (11, 2) = (0, 3)$

Bob: $K = bA = 6 \ast (3, 8) = (0, 3)$

Console:

```

Alice's Shared Point:
(0, 3)
Bob's Shared Point:
(0, 3)

```

```
from tinyec.ec import SubGroup, Curve
```

```
field = SubGroup(p=13, g=(0, 3), n=17, h=1)
curve = Curve(a=5, b=9, field=field,
name='p1707')
```

```
print('curve:', curve)
```

```
for k in range(0, 24):
    p = k * curve.g
    print(f"{k} * G = ({p.x}, {p.y})")
```

Using tinyec library to show the points within the subgroup G,

where n is the order and h can be ignored for this demo

```
G = (0, 3)
```

```
# b) Alice's public key
```

```
G2 = pointDoubling(G[0], G[1], 5, 13)
G3 = pointAddition(G2[0], G2[1], 0, 3, 13)
print("Alice's Public Point: \t", G3)
```

```
# c) Bobs public key
```

```
G6 = pointDoubling(G3[0], G3[1], 5, 13)
print("Bob's Public Point: \t", G6)
```

```
# d)
```

```
Alice computes shared Key
```

```
a = 3
B = (11, 2)
B2 = pointDoubling(B[0], B[1], 5, 13)
B3 = pointAddition(B[0], B[1], B2[0], B2[1], 13)
print("Alice's Shared Point: \t", B3)
```

```
# Bob computes shared Key
```

```
b = 6
A = (3, 8)
A2 = pointDoubling(A[0], A[1], 5, 13)
A3 = pointAddition(A[0], A[1], A2[0], A2[1], 13)
A6 = pointDoubling(A3[0], A3[1], 5, 13)
print("Bob's Shared Point: \t", A6)
```